

# Documentation d'Architecture

Renote — Migration server-driven → client-driven

---

Projet P3 — OpenClassrooms  
PHP 8.4 / Laravel 12 · React 18 · Zustand · Sanctum · SQLite

# 1. Introduction

Renote est une application web de prise de notes permettant à un utilisateur de créer, visualiser et organiser ses notes à l'aide de tags. L'objectif de cette refonte est de faire évoluer l'application d'une architecture monolithique server-driven vers une architecture client-driven découplée, afin de supporter plusieurs interfaces clientes (web, mobile) et d'exposer une API REST consommable par des services externes.

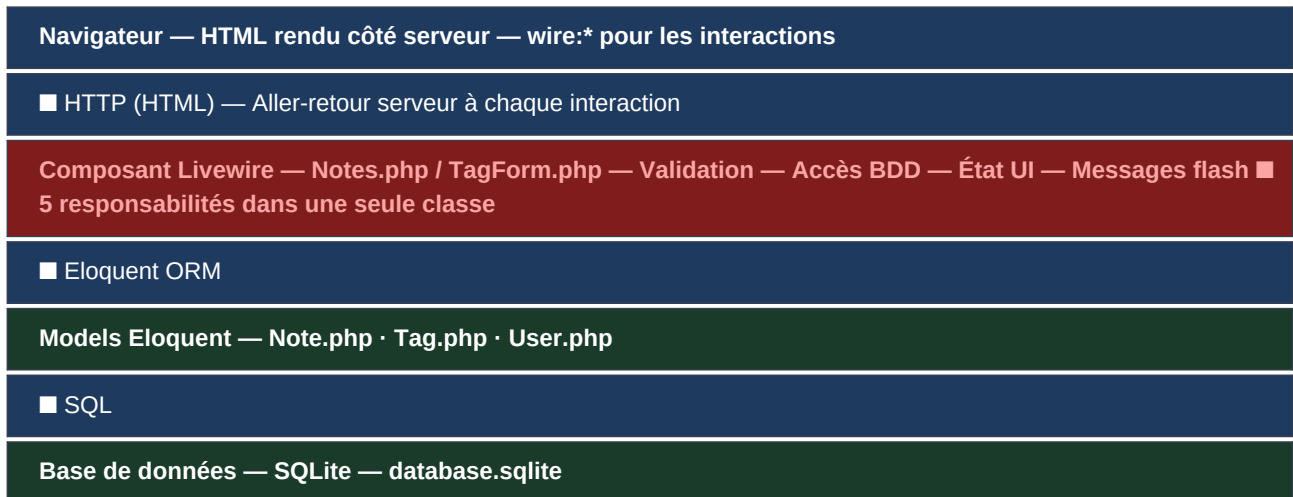
Cette documentation décrit les quatre transformations réalisées :

| Transformation                          | Description                                                                                                                                                               |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Migration server-driven → client-driven | Le HTML n'est plus généré par PHP. React gère l'affichage côté navigateur. Le back-end PHP répond exclusivement en JSON.                                                  |
| Passage au pattern MVC                  | Le code est réorganisé en Controllers (HTTP), Services (logique métier) et Models (accès données). Chaque couche a une responsabilité unique (principe S de SOLID).       |
| Découplage complet front / back         | Le backend (backend/) est une API PHP pure sans vues. Le frontend (frontend/) est une app React standalone avec Vite. Les deux communiquent exclusivement via l'API REST. |
| Authentification sans état via Sanctum  | Laravel Sanctum génère des tokens Bearer stockés en BDD (personal_access_tokens). Plus de cookies de session ni de CSRF — compatible Postman, mobile et web.              |
| Ajout d'un state management côté front  | Zustand gère l'état global partagé entre composants React (notes, tags, token, loading, error) selon le pattern Flux.                                                     |

## 2. Architecture de base

### 2.1 Diagramme de composants (architecture de base)

L'architecture initiale est entièrement server-driven. Livewire génère le HTML côté serveur PHP et gère les interactions via WebSockets PHP. Il n'y a aucune séparation entre le front et le back.



■ En rouge : le composant Livewire qui cumule toutes les responsabilités.

### 2.2 Points faibles constatés

| Point faible              | Description                                                                                                                                              |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Couplage fort             | Les composants Livewire (Notes.php, TagForm.php) mélangent validation, accès BDD, état UI et gestion des messages. Aucune séparation de responsabilités. |
| Absence d'API             | routes/web.php ne contient que des routes HTML. Aucun endpoint /api/* n'existe. Le front est indissociable du back.                                      |
| Architecture non scalable | Livewire génère le HTML côté serveur. Il est impossible d'avoir une app mobile ou un second client web sans refactorisation complète.                    |
| Duplication de logique    | Tag::all() est appelé dans mount() ET refreshTags() dans Notes.php — même requête dupliquée. Violé le principe DRY.                                      |
| Testabilité nulle         | La logique métier est dans les composants Livewire. Il est impossible de tester create() ou delete() sans démarrer Laravel entier.                       |

## 3. Architecture cible

### 3.a Composants d'architecture

L'architecture cible sépare complètement le front-end React du back-end PHP/Laravel. La communication entre les deux se fait exclusivement via l'API REST (HTTP/JSON). Côté back-end, le pattern MVC est appliqué avec une couche Service supplémentaire.

**FRONT-END standalone (frontend/ — React + Zustand) — Login · Register · Dashboard · NoteForm · NoteList · TagForm · services/api.js**

■ HTTP REST / JSON (axios + Authorization: Bearer {token})

**routes/api.php (backend/) — 8 endpoints REST · Middleware auth:sanctum · Préfixe /api**

■ Laravel Router

**Controllers (couche HTTP) — NoteController.php · TagController.php Reçoit la requête, valide, délègue au Service, renvoie JSON**

■ Injection de dépendances (principe D)

**Services (couche métier) — NoteService.php · TagService.php Logique métier pure — réutilisable et testable indépendamment**

■ Eloquent ORM

**Models (couche données) — Note.php · Tag.php · User.php · Relations belongsTo / hasMany**

■ PDO / SQL

**Base de données SQLite — database/database.sqlite**

### Protocoles de communication

| Lien                 | Protocole      | Format      | Détail                                           |
|----------------------|----------------|-------------|--------------------------------------------------|
| React → API Laravel  | HTTP REST      | JSON        | axios + header Authorization: Bearer {token}     |
| API → Controller     | Laravel Router | PHP interne | Middleware auth:sanctum vérifie le token Sanctum |
| Controller → Service | Injection DI   | PHP interne | Laravel injecte le Service automatiquement       |
| Service → Model      | Eloquent ORM   | PHP interne | Note::create(), Note::where(), Tag::all()        |
| Model → BDD          | PDO / SQL      | SQLite      | Fichier database/database.sqlite                 |

## 3.b Spécifications API REST

Tous les endpoints sont préfixés par /api. Les endpoints protégés nécessitent le header : Authorization: Bearer {token}. Format de réponse uniforme : { "status": "success"|"error", "data": ..., "message": ... }

### Endpoints — Authentification (publics)

|              |                                                                                                                                             |                                                           |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <b>POST</b>  | /api/register                                                                                                                               | 201 Created — 422 Email déjà utilisé / validation échouée |
| Description  | Crée un compte utilisateur et retourne un token Sanctum. Le token est stocké dans la table personal_access_tokens.                          |                                                           |
| Requête      | <pre>{   "name": "Mon Nom",   "email": "mon@email.fr",   "password": "password123" }</pre>                                                  |                                                           |
| Réponse JSON | <pre>{   "status": "success",   "token": "1 nJCMiV5betpU045g...",   "user": { "id": 1, "name": "Mon Nom", "email": "mon@email.fr" } }</pre> |                                                           |

|              |                                                                                                                                         |                                     |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| <b>POST</b>  | /api/login                                                                                                                              | 200 OK — 401 Identifiants invalides |
| Description  | Authentifie un utilisateur et retourne un nouveau token Sanctum. Les anciens tokens sont supprimés.                                     |                                     |
| Requête      | <pre>{   "email": "mon@email.fr",   "password": "password123" }</pre>                                                                   |                                     |
| Réponse JSON | <pre>{   "status": "success",   "token": "2 FVZYUcYfMzMw...",   "user": { "id": 1, "name": "Mon Nom", "email": "mon@email.fr" } }</pre> |                                     |

|              |                                                                                             |                              |
|--------------|---------------------------------------------------------------------------------------------|------------------------------|
| <b>POST</b>  | /api/logout                                                                                 | 200 OK — 401 Non authentifié |
| Description  | Révoque le token actuel (le supprime de personal_access_tokens). Nécessite le Bearer token. |                              |
| Requête      | Aucun body. Header requis : Authorization: Bearer {token}                                   |                              |
| Réponse JSON | <pre>{   "status": "success",   "message": "Déconnecté." }</pre>                            |                              |

### Endpoints — Notes (protégés — Bearer token requis)

|             |                                                                                             |                              |
|-------------|---------------------------------------------------------------------------------------------|------------------------------|
| <b>GET</b>  | /api/notes                                                                                  | 200 OK — 401 Non authentifié |
| Description | Retourne toutes les notes de l'utilisateur connecté, avec leur tag associé (eager loading). |                              |
| Requête     | Aucun body. Header : Authorization: Bearer {token}                                          |                              |

|                 |                                                                                                                                                                                                          |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Réponse<br>JSON | <pre>{   "status": "success",   "data": [     {       "id": 1,       "text": "Ma note",       "tag": { "id": 2, "name": "Travail" },       "created_at": "2025-07-16T20:00:00.000000Z"     }   ] }</pre> |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                 |                                                                                                                                                                                                       |                                                            |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>POST</b>     | /api/notes                                                                                                                                                                                            | 201 Created — 401 Non authentifié — 422 Validation échouée |
| Description     | Crée une nouvelle note pour l'utilisateur connecté.                                                                                                                                                   |                                                            |
| Requête         | <pre>{   "text": "Contenu de la note",   "tag_id": 1 }</pre>                                                                                                                                          |                                                            |
| Réponse<br>JSON | <pre>{   "status": "success",   "message": "Note créée.",   "data": {     "id": 5,     "text": "Contenu de la note",     "tag": { "id": 1, "name": "Personnel" },     "created_at": "..."   } }</pre> |                                                            |

|                 |                                                                                                                                                                                                        |                                                                  |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>DELETE</b>   | /api/notes/{id}                                                                                                                                                                                        | 200 OK — 401 Non authentifié — 404 Note introuvable/accès refusé |
| Description     | Supprime une note. Vérifie que la note appartient à l'utilisateur connecté.                                                                                                                            |                                                                  |
| Requête         | Aucun body. L'id est dans l'URL.                                                                                                                                                                       |                                                                  |
| Réponse<br>JSON | <pre>{   "status": "success",   "message": "Note supprimée." }</pre><br><pre>// Si la note n'appartient pas à l'user : {   "status": "error",   "message": "Note introuvable ou accès refusé." }</pre> |                                                                  |

**Endpoints — Tags (protégés — Bearer token requis)**

|             |                                                                               |                              |
|-------------|-------------------------------------------------------------------------------|------------------------------|
| <b>GET</b>  | /api/tags                                                                     | 200 OK — 401 Non authentifié |
| Description | Retourne la liste de tous les tags disponibles (non filtrés par utilisateur). |                              |
| Requête     | Aucun body. Header : Authorization: Bearer {token}                            |                              |

|                 |                                                                                                                                |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------|
| Réponse<br>JSON | <pre>{   "status": "success",   "data": [     { "id": 1, "name": "Personnel" },     { "id": 2, "name": "Travail" }   ] }</pre> |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------|

|                 |                                                                                                                                                                                                                                |                                                                                |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>POST</b>     | /api/tags                                                                                                                                                                                                                      | 201 Created — 401 Non authentifié — 422 Validation (nom dupliqué ou trop long) |
| Description     | Crée un nouveau tag. Le nom doit être unique (max 50 caractères).                                                                                                                                                              |                                                                                |
| Requête         | <pre>{   "name": "Travail" }</pre>                                                                                                                                                                                             |                                                                                |
| Réponse<br>JSON | <pre>{   "status": "success",   "message": "Tag créé.",   "data": { "id": 3, "name": "Travail" } }</pre> <p>// Si le nom existe déjà :</p> <pre>{   "status": "error",   "message": "The name has already been taken." }</pre> |                                                                                |

### 3.c Data flow du state management

Zustand applique le pattern Flux : les actions modifient le store, les composants qui s'y abonnent se re-rendent automatiquement. Ci-dessous le flux complet pour la création d'une note.

#### Flux actions / events → store → UI

| Type   | Action                                    | Détail technique                                                                         |
|--------|-------------------------------------------|------------------------------------------------------------------------------------------|
| EVENT  | Utilisateur soumet le formulaire          | NoteForm.jsx — handleSubmit(e) déclenché par onSubmit du                                 |
| ACTION | addNote(text, tag_id) appelée             | Le composant appelle addNote() du store Zustand — aucun appel fetch direct               |
| EFFET  | Appel API POST /api/notes                 | useAppStore : createNote(text, tag_id) → api.js → axios.post('/notes', {...})            |
| EFFET  | Laravel traite la requête                 | NoteController.store() → NoteService.create() → Note::create() → SQLite                  |
| EFFET  | Réponse JSON 201                          | { "status": "success", "data": { id, text, tag, created_at } }                           |
| STORE  | set({ notes: [newNote, ...state.notes] }) | Zustand met à jour l'état global — optimistic : on préfixe en tête de liste              |
| UI     | NoteList.jsx se re-rend automatiquement   | Zustand notifie NoteList car il s'est abonné à state.notes via useAppStore(s => s.notes) |

#### Gestion des erreurs et état de chargement

Le store Zustand centralise les états loading et error. Tout appel API commence par set({ loading: true, error: null }) et se termine soit par set({ loading: false }) en succès, soit par set({ error: message, loading: false }) en échec. Les composants lisent ces valeurs et affichent un spinner ou un message d'erreur.



## 4. Analyse de l'écart

Liste des composants ajoutés, modifiés et supprimés lors de la migration.

### Composants ajoutés

| Fichier                                             | Type            | Rôle                                                             |
|-----------------------------------------------------|-----------------|------------------------------------------------------------------|
| backend/app/Services/NoteService.php                | Service PHP     | Logique métier notes — getAllForUser(), create(), delete()       |
| backend/app/Services/TagService.php                 | Service PHP     | Logique métier tags — getAll(), create()                         |
| backend/app/Http/Controllers/Api/NoteController.php | Controller PHP  | Gère les requêtes HTTP API notes (index, store, destroy)         |
| backend/app/Http/Controllers/Api/TagController.php  | Controller PHP  | Gère les requêtes HTTP API tags (index, store)                   |
| backend/app/Http/Controllers/Api/AuthController.php | Controller PHP  | Register, Login, Logout via Sanctum Bearer token                 |
| backend/routes/api.php                              | Routes Laravel  | 8 endpoints REST — auth publics + notes/tags protégés            |
| frontend/src/main.jsx                               | Entrée React    | Monte dans le div#root de index.html                             |
| frontend/src/App.jsx                                | Composant React | Routing Login/Register/Dashboard selon le token Zustand          |
| frontend/src/components/Login.jsx                   | Composant React | Page connexion — appelle store.login()                           |
| frontend/src/components/Register.jsx                | Composant React | Page inscription — appelle store.register()                      |
| frontend/src/components/Dashboard.jsx               | Composant React | Page principale avec sidebar — charge les tags au montage        |
| frontend/src/components/NoteForm.jsx                | Composant React | Formulaire création note — lit le store, appelle addNote()       |
| frontend/src/components/NoteList.jsx                | Composant React | Liste notes — charge via fetchNotes(), appelle removeNote()      |
| frontend/src/components/TagForm.jsx                 | Composant React | Formulaire création tag — appelle addTag()                       |
| frontend/src/store/useAppStore.js                   | Store Zustand   | État global (token, user, notes, tags, loading, error) + actions |
| frontend/src/services/api.js                        | Service JS      | Couche API — Bearer token auto via intercepteur axios            |

### Composants modifiés

| Fichier | Modification |
|---------|--------------|
|---------|--------------|

|                                 |                                                                                              |
|---------------------------------|----------------------------------------------------------------------------------------------|
| backend/bootstrap/app.php       | Ajout de api: routes/api.php dans withRouting() — suppression des middlewares de session web |
| backend/composer.json           | Ajout de laravel/sanctum — suppression de livewire/flux et livewire/volt                     |
| backend/app/Models/User.php     | Ajout du trait HasApiTokens (Sanctum) pour createToken() et tokens()                         |
| backend/bootstrap/providers.php | Suppression de VoltServiceProvider (Livewire supprimé)                                       |
| frontend/package.json           | Ajout de react, react-dom, zustand, axios, tailwindcss, @vitejs/plugin-react                 |
| frontend/vite.config.js         | Configuration Vite standalone avec proxy /api → localhost:8000 pour éviter le CORS           |

## Composants supprimés

| Fichier supprimé                         | Remplacé par                                                       |
|------------------------------------------|--------------------------------------------------------------------|
| app/Livewire/Notes.php                   | NoteController.php + NoteService.php + NoteList.jsx + NoteForm.jsx |
| app/Livewire/TagForm.php                 | TagController.php + TagService.php + TagForm.jsx                   |
| app/Livewire/Actions/Logout.php          | AuthController.php → POST /api/logout                              |
| resources/views/ (toutes les vues Blade) | Composants React dans frontend/src/components/                     |
| routes/web.php + routes/auth.php         | routes/api.php uniquement — le backend est une API pure sans HTML  |
| livewire/flux + livewire/volt (packages) | laravel/sanctum pour l'authentification par token Bearer           |

## 5. Justification de l'approche architecturale

### 5.1 Choix de Zustand pour le state management

Zustand a été choisi pour sa simplicité et son faible volume de code. Redux Toolkit aurait été une alternative valide mais impose des conventions plus lourdes (slices, reducers, dispatch) inadaptées à un projet de cette taille. MobX aurait introduit le pattern MVVM avec des observables automatiques, mais sa courbe d'apprentissage est plus élevée. Le Context API natif React aurait causé des re-renders excessifs.

Zustand implémente le pattern Flux de manière simplifiée : les actions (addNote, removeNote, addTag) modifient directement le store via set(). Les composants s'abonnent avec des sélecteurs ciblés (useAppStore(s => s.notes)) et ne se re-rendent que lorsque leur portion de l'état change.

### 5.2 Principes SOLID appliqués

| Principe                  | Application dans Renote                                                                                                                                                                                                   |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S — Single Responsibility | Chaque classe a une seule responsabilité : NoteController gère HTTP, NoteService contient la logique métier, Note (Model) accède à la BDD. Côté React, chaque composant gère une seule partie de l'UI.                    |
| O — Open/Closed           | Ajouter un nouvel endpoint (ex. PUT /api/notes/{id}) ne nécessite pas de modifier NoteService ou les autres contrôleurs — on ajoute une méthode.                                                                          |
| L — Liskov Substitution   | NoteController extends Controller sans surcharger de comportements critiques. Non directement observable ici faute d'héritage métier complexe.                                                                            |
| I — Interface Segregation | NoteController n'expose que les routes liées aux notes. TagController n'expose que les routes tags. Aucun contrôleur 'fourre-tout'.                                                                                       |
| D — Dependency Inversion  | NoteController ne fait pas new NoteService(). Il déclare sa dépendance dans le constructeur et Laravel l'injecte automatiquement. Côté React, les composants dépendent de api.js (abstraction), pas de fetch directement. |

### 5.3 Autres décisions techniques

| Décision                                      | Justification                                                                                                                                                                                                                                                                                                        |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Séparation des responsabilités (SoC)          | La validation est faite dans le Controller (frontière système), pas dans le Service. Le Service est ainsi réutilisable sans valider deux fois.                                                                                                                                                                       |
| Sanctum Bearer token pour l'authentification  | Sanctum génère des tokens opaques stockés en BDD (personal_access_tokens). Le token est retourné dans la réponse JSON après login/register. React le stocke en mémoire (Zustand) + localStorage et l'envoie via Authorization: Bearer. Compatible Postman, mobile et tout client HTTP — aucun cookie ni CSRF requis. |
| Séparation backend/ et frontend/ dans le repo | Le backend est une API PHP pure (aucune vue, aucun asset front). Le frontend est une app React standalone avec Vite sur port 5174. Les deux peuvent être déployés indépendamment sur des serveurs différents.                                                                                                        |
| SQLite comme base de données                  | Base embarquée, zéro configuration. Facilement remplaçable par MySQL/PostgreSQL en changeant DB_CONNECTION dans .env — Laravel Eloquent abstrait le moteur SQL.                                                                                                                                                      |

|                                 |                                                                                                                                                                |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Format de réponse JSON uniforme | { status, data, message } pour toutes les réponses. Facilite la gestion côté React : on sait toujours où chercher les données.                                 |
| Optimistic update dans le store | Après création/suppression, on met à jour le store local sans re-fetcher toute la liste. Cela évite un aller-retour réseau inutile et rend l'UI plus réactive. |